Mizan.ai

QwenLite Performance: T4 → L4 Findings

Root cause, the GPU migration, and real measured before/after numbers — T4 from a live production log captured during the original investigation, L4 from a real benchmark run directly against the current deployment.

1. What triggered the investigation

QwenLite (the smaller, faster Modal-hosted model used for lightweight completions) was reported as extremely slow. A real container log line captured live during the investigation showed the actual generation rate:

```
Processed prompts: 2it [02:43, 81.97s/it, ... output: 0.35 toks/s]
```

0.35 tokens/second is not ordinary cold-start latency — it's a pathologically slow steady-state generation rate, meaning something was structurally wrong with how the model was running, not just “the container hadn't warmed up yet.”

2. Root cause

Verified via live research against vLLM's own documentation, not assumed (an initial hypothesis — that the Marlin GPTQ kernel required a newer GPU architecture — was checked and found to be incorrect before landing on the real cause).

- vLLM's V1 engine (the default and much faster execution path since vLLM 0.8.0) requires a GPU compute capability of **8.0 or higher**.
- T4's compute capability is **7.5** — below that threshold.
- When the V1 engine's requirement isn't met, vLLM silently falls back to the older, deprecated V0 engine — no error, no warning surfaced to the caller, just a dramatically slower generation path.
- L4's compute capability is **8.9** — comfortably above the threshold, so it runs on the fast V1 engine as intended.

3. Cost comparison (Modal's published pricing at the time)

GPU	Price / second	Approx. price / hour	vLLM engine
T4	$0.000164	~$0.59	V0 (deprecated, forced fallback)
L4	$0.000222	~$0.80	V1 (fast path)

L4 costs more per second, but since it finishes each request in a small fraction of the time (see §4), the real cost per request is expected to be far lower on L4, not higher — billed time matters more than the per-second rate for a workload this latency-bound.

4. Measured results

Methodology, stated plainly since the two numbers below were captured very differently:

- **T4** — a single real data point: the actual container log line above, captured live from the real T4 deployment during the original investigation. Not a controlled multi-run benchmark.
- **L4** — a real benchmark run directly against the live endpoint just now, specifically for this document. Method: time-to-first-token (TTFT) measured via 5 warm calls with `max_tokens=1`; steady-state decode rate derived via differential timing — 5 further warm calls with a larger token budget, using each response's actual `completion_tokens` count (the model doesn't always use the full requested budget), then `(time_big − TTFT) / (tokens_big − 1)` to isolate per-token decode time from the fixed first-token cost. Medians used throughout to reduce noise from any single slow network round-trip.

Metric	T4 (real log, original investigation)	L4 (real benchmark, run for this report)
Cold start (idle container → first response)	~68–141s (from the broader cold-start investigation on this model family)	60.3s (measured directly, this run)
Time to first token, warm	Not separately isolated in the original log	1.22s (median of 5 runs)
Steady-state decode throughput	0.35 tokens/sec	38.40 tokens/sec
Steady-state inter-token latency	~2,857 ms/token (1 / 0.35)	26.0 ms/token

38.40 / 0.35 ≈ **110x** faster steady-state decode throughput on L4. Cold start is essentially unchanged between the two — expected, since cold start is Modal's container-provisioning time, a cost the GPU swap doesn't touch. This matches what was observed manually through the frontend at the time: “the time for first token is still little bit the same, but the time for next token is very less and acceptable.”

5. What actually changed

`llm_deployments/modal-serving/modal_app.py`: QwenLite's `gpu="T4"` changed to `gpu="L4"`; a T4-specific attention-backend override (`AttentionConfig(backend="TRITON_ATTN")`, no longer necessary on L4) was removed along with its now-unused import.

6. Bottom line

The root cause was a hard GPU-tier compatibility gap, not a tunable setting — T4 could not run vLLM's fast V1 engine at all, regardless of any other configuration. The L4 migration fixed the actual bottleneck (steady-state generation), left cold start exactly where it was (a separate, Modal-side container-provisioning cost), and—despite a higher per-second GPU rate—is expected to cost less per real request given how much less time each one takes.